System Overview

Workgraph is a task coordination system for humans and AI agents. It models work as a directed graph: tasks are nodes, dependency edges connect them, and a scheduler moves through the structure by finding what is ready and dispatching agents to do it. Everything—the graph, the agent identities, the configuration—lives in plain files under version control. There is no database. There is no mandatory server. The simplest possible deployment is a directory and a command-line tool.

But simplicity of storage belies richness of structure. The graph is not a flat list. Dependencies create ordering, parallelism emerges from independence, and structural cycles introduce intentional iteration where work revisits earlier stages. Layered on top of this graph is an *agency*—a system of composable identities that gives each agent a declared purpose and a set of constraints. Together, the graph and the agency form a coordination system where the work is precisely defined, the workers are explicitly characterized, and improvement is built into the process.

This section establishes the big picture. The details follow in later sections: the task graph in *Section 2*, the agency model in *Section 3*, coordination and execution in *Section 4*, and evolution in *Section 5*.

The Graph Is the Work

A *task* is the fundamental unit of work in workgraph. Every task has an ID, a title, a status, and may carry metadata: estimated hours, required skills, deliverables, inputs, tags. Tasks are the atoms. Everything else—dependencies, scheduling, dispatch—is structure around them.

Tasks are connected by *dependency* edges expressed through the `after` field. If task B lists task A in its `after` list, then B comes after A—it cannot begin until A reaches a *terminal* status, that is, until A is done, failed, or abandoned. This is a deliberate choice: all three terminal statuses unblock dependents, because a failed upstream task should not freeze the entire graph. The downstream task gets dispatched and can decide what to do with a failed predecessor.

From these simple rules, complex structures emerge. A single task blocking several children creates a fan-out pattern—parallel work radiating from a shared prerequisite. Several tasks blocking one aggregator create a fan-in—convergence into a synthesis step. Linear chains form pipelines. These are not built-in primitives. They arise naturally from dependency edges, the way sentences arise from words.

The graph is also not required to be acyclic. *Structural cycles*—cycles that form naturally in the `after` edges—enable intentional iteration. A write-review-revise cycle, a CI retry pipeline, a monitoring loop: all are expressible as dependency chains where an `after` edge points backward, creating a cycle detected automatically by the system. The cycle's header task carries a `CycleConfig` with a mandatory `max_iterations` cap and an optional *guard* condition. The header receives a back-edge exemption in the readiness check, allowing the cycle to start and iterate. When all cycle members complete and the guard is satisfied, the entire cycle re-opens for the next iteration. When iterative work reaches a stable state before exhausting its iteration budget, any agent in the cycle can signal convergence to halt early.

The entire graph lives in a single JSONL file—one JSON object per line, human-readable, friendly to version control, protected by file locking for concurrent writes. This is the canonical state. Every command reads from it; every mutation writes to it.

The Agency Is Who Does It

Without the agency system, every AI agent dispatched by workgraph is a blank slate—a generic assistant that receives a task description and does its best. This works, but it leaves performance on

the table. A generic agent has no declared priorities, no persistent personality, no way to improve across tasks. The agency system addresses this by giving agents *composable identities*.

An identity has two components. A *role* defines *what* the agent does: its description, its skills, its desired outcome. A *motivation* defines *why* the agent acts the way it does: its priorities, its acceptable trade-offs, and its hard constraints. The same role paired with different motivations produces different agents. A Programmer role with a Careful motivation—one that prioritizes reliability and rejects untested code—will behave differently than the same Programmer role with a Fast motivation that tolerates rough edges in exchange for speed. The combinatorial identity space is the key insight: a handful of roles and motivations yield a diverse population of agents.

Each role, each motivation, and each agent is identified by a *content-hash ID*—a SHA-256 hash of its identity-defining fields, displayed as an eight-character prefix. Content-hashing gives three properties that matter: identity is deterministic (same content always produces the same ID), deduplicating (you cannot create two identical entities), and immutable (changing an identity-defining field produces a *new* entity; the old one remains). This makes identity a mathematical fact, not an administrative convention. You can verify that two agents share the same role by comparing hashes.

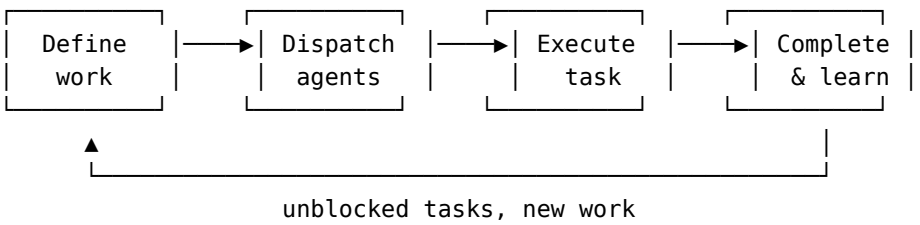
When an agent is dispatched to a task, its role and motivation are resolved—skills fetched from files, URLs, or inline definitions—and injected into the prompt. The agent doesn't just receive a task description; it receives an identity. This is what separates a workgraph agent from a one-off LLM call.

Human agents participate in the same model. The only difference is the *executor*: AI agents use `claude` (or another LLM backend); human agents use `matrix`, `email`, `shell`, or another human-facing channel. Human agents don't need roles or motivations—they bring their own judgment. But both human and AI agents are tracked, evaluated, and coordinated uniformly. The system does not distinguish between them in its bookkeeping; only the dispatch mechanism differs.

Because identities are content-hashed, they travel well. Agency entities—roles, motivations, and their evaluation histories—can be shared across projects through federation, carrying lineage and performance data intact. A proven architect role in one project can be pulled into another without re-creation; the content-hash guarantees it is the same entity everywhere.

The Core Loop

Workgraph operates through a cycle that applies at every scale, from a single task to a multi-week project:



Listing 1: The heartbeat of a workgraph project.

Define work. Add tasks to the graph with their dependencies, skills, deliverables, and time estimates. The graph is the plan. Modifying it is cheap—add a task, change a dependency, split a bloated task into subtasks. The graph adapts as understanding evolves.

Dispatch agents. A *coordinator*—the scheduling brain inside an optional service daemon—finds *ready* tasks: those that are open, not paused, past any time constraints, and whose every dependency

has reached a terminal status. For each ready task, it resolves the executor, builds context from completed dependencies, renders the prompt with the agent's identity, and spawns a detached process. The coordinator *claims* the task before spawning to prevent double-dispatch.

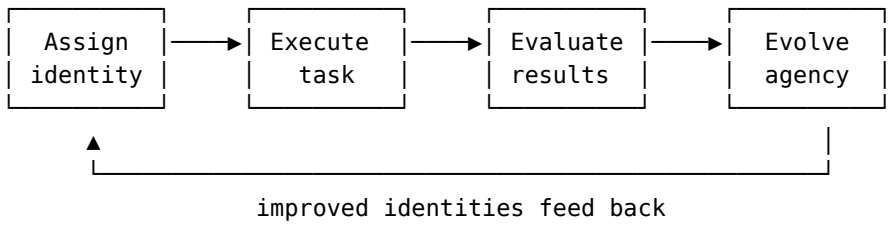
Execute. The spawned agent does its work. It may log progress, record artifacts, create subtasks, or mark the task done or failed. It operates with full autonomy within the boundaries set by its role and motivation.

Complete and learn. When a task reaches a terminal status, its dependents may become ready, continuing the flow. If the agency system is active, a completed task can also trigger *evaluation*—a scored assessment across four dimensions (correctness, completeness, efficiency, style adherence) whose results propagate to the agent, its role, and its motivation.

This is the basic heartbeat. Most projects run on this loop alone.

The Agency Loop

The agency system extends the core loop with a second, slower cycle of improvement:



Listing 2: The agency improvement cycle.

Assign identity. Before a task is dispatched, an agent identity is bound to it—either manually or through an auto-assign system where a dedicated assigner agent evaluates the available agents and picks the best fit. *Assignment* sets identity; it is distinct from *claiming*, which sets execution state.

Execute task. The agent works with its assigned identity injected into the prompt.

Evaluate results. After the task completes, an evaluator agent scores the work. Evaluation produces a weighted score that propagates to three levels: the agent, its role (with the motivation as context), and its motivation (with the role as context). This three-level propagation creates the data needed for cross-cutting analysis—how does a role perform with different motivations, and vice versa?

Evolve the agency. When enough evaluations accumulate, an evolver agent analyzes performance data and proposes structured changes: mutate a role to strengthen a weak dimension, cross two high-performing roles into a hybrid, retire a consistently poor motivation, create an entirely new role for unmet needs. Modified entities receive new content-hash IDs with *lineage* metadata linking them to their parents, creating an auditable evolutionary history. Evolution is a manual trigger (`wg evolve`), not an automated process, because the human decides when there is enough data to act on and reviews every proposed change.

Each step in this cycle can be manual or automated. A project might start with manual assignment and no evaluation, graduate to auto-assign once agent identities stabilize, enable auto-evaluate to build a performance record, and eventually run evolution to refine the agency. The system meets you where you are.

How They Relate

The task graph and the agency are complementary systems with a clean separation. The graph defines *what* needs to happen and *in what order*. The agency defines *who* does it and *how they approach it*. Neither depends on the other for basic operation: you can run workgraph without the

agency (every agent is generic), and you can define agency entities without a graph (though they have nothing to do). The power is in the combination.

The coordinator sits at the intersection. It reads the graph to find ready work, reads the agency to resolve agent identities, dispatches the work, and—when evaluation is enabled—closes the feedback loop by scoring results and feeding data back into the agency. The graph is the skeleton; the agency is the musculature; the coordinator is the nervous system.

Workgraph is not a closed system. External tools—CI pipelines, portfolio trackers, peer organizations—can observe the graph through a real-time event stream and inject information back through several channels: recording evaluations with external source tags, importing trace data from peers, adding tasks, or updating state directly. Each task carries a *visibility* field (`internal`, `public`, or `peer`) that controls what information crosses organizational boundaries when traces are exported. This boundary discipline makes collaboration possible without exposing internal deliberation.

Everything is files. The graph is JSONL. Agency entities—roles, motivations, agents—are YAML. Configuration is TOML. Evaluations are YAML. Underneath it all, an operations log records every mutation to the graph—the project's trace. This trace is organizational memory: queryable for provenance, exportable for cross-boundary sharing with visibility filtering, and extractable into parameterized workflow templates that capture proven patterns for reuse. There is no database, no external dependency, no required network connection. The optional service daemon automates dispatch but is not required for operation. You can run the entire system from the command line, one task at a time, or you can start the daemon and let it manage a fleet of parallel agents. The architecture scales from a solo developer tracking personal tasks to a coordinated multi-agent project with dozens of concurrent workers, all from the same set of files in a `.workgraph` directory.